Canadian Wildfires in 2023

Selection of relevant Canadian Cities

This notebook selects a representative set of Canadian cities out of the World Cities dataset. Of each province the capital is being included (admin & primary). To fill up the 40 relevant cities an algorithm was computed to add the most populated cities of Canada. Since in Canada there is a huge difference between high populated and a low populated City an additional algorithm was included to only select a city if its at least 150km apart from any other selected city

Short summary of the following steps

0. Import & Constants
1. Preparation and Cleaning
2. City Selection
3. Export of processed data

0. Import & Constants

```
from pathlib import Path

import pandas as pd
import geopandas as gpd

#With pathlib the storage paths stay portable across any device
#Using ../ we go up one level (out of notbook folder) to reach the
#project root and then enter the data folder

DATA_DIR = Path("../data")
RAW_PATH       = DATA_DIR / "raw"       / "worldcities.csv"
PROCESSED_PATH = DATA_DIR / "processed" / "selected_cities_canada.csv"
```

a) Analysis parameters

```
#Coordinate reference systems
#EPSG: 4326 is being used as a default in the base data (geographic CRS (in degrees))
#EPSG: 3347 is the metric projection for Canada, required for calculations in meters

CRS_DEFAULT_DEGREE = "EPSG: 4326"
CRS_COUNTRY_METRIC = "EPSG: 3347"


#storing the relevant City as a constante
TARGET_COUNTRY = "Canada"

#Cities which are always being included regardless of spacing
CAPITAL_CITIES = ["primary", "admin"]
```

```
#Number of Cities which have to be selected
MAX_CITIES = 40


#Minimum allowed distance between cities in meters.
#Any other City closer than this distance is skipped to avoid spatial clustering

MIN_SPACE_KM = 150
```

b) Load raw data

```
cities_df = pd.read_csv(RAW_PATH, sep=",")

assert len(cities_df) > 0, "Loaded file is empty."
print(f"Loaded {len(cities_df)} cities worldwide from raw data.")
```

```
Loaded 48059 cities worldwide from raw data.
```

1. Preparation and Cleaning

a) Filter

```
#filtering to target country

cities_country_df = cities_df[
    cities_df["country"] == TARGET_COUNTRY].copy()

assert len(cities_country_df) > 0, f"No cities found in '{TARGET_COUNTRY}'."
print(f"{TARGET_COUNTRY} cities in raw data: {len(cities_country_df)}")
```

```
Canada cities in raw data: 485
```

b) CRS transformation

```
#tansforming the data into a GeoDataFrame in geographic CRS
#Using panda we need a transformation because otherwise latitude and
#longitude are simple numbers and not recognized as a geometry

cities_country_gdf = gpd.GeoDataFrame(
    cities_country_df,
    geometry=gpd.points_from_xy(
        cities_country_df["lng"],
        cities_country_df["lat"]),
    crs = CRS_DEFAULT_DEGREE
)

#For a metric calculation such as a distance the data needs to be
```

```
#stored in a metric crs (CRS_COUNTRY_METRIC)
#If you change the "TARGET_COUNTRY" you have to change this
#CRS_COUNTRY_METRIC as well

cities_country_gdf = cities_country_gdf.to_crs(CRS_COUNTRY_METRIC)

assert cities_country_gdf.crs is not None, "Reprojection failed"
print(f"GeoDataFrame CRS: {cities_country_gdf.crs.to_epsg()}")
```

```
GeoDataFrame CRS: 3347
```

2. City selection

a) Preparation of inputs

```
#Capital and admin cities are always included no matter the population.
primary_cities = cities_country_gdf[
    cities_country_gdf["capital"].isin(CAPITAL_CITIES)]

#All the others are possible candidates for the spacing-based selection
other_cities = cities_country_gdf[
    ~cities_country_gdf.index.isin(primary_cities.index)]

#By sorting the cities we make sure, that the most populated places
#fill the open slots in the list.
other_cities = other_cities.sort_values(
    "population",
    ascending=False
).reset_index(drop=True)
```

b) Definition Selection Loop

```
def select_cities_by_space(
    seed_gdf: gpd.GeoDataFrame,
    candidates_gdf: gpd.GeoDataFrame,
    min_spacing_m: float,
    max_cities: int,
) -> gpd.GeoDataFrame:
    """
    Adds possible cities to a predefined selection
    of cities if they are spaced sufficiently.

    It iterates through candidates with the largest population first.
    Then it includes the city only when the distance to each already
    selected city is greater than min_spacing_m. This assumption was
    made to prevent that multiple urban agglomeration from the same
    city are being selected. Finally it stops when max_cities is
    reached so that not every city in the country is on the final map.
```

```
    Parameters:
    -----------
    seed_gdf: gpd.GeoDataFrame
        In this case the Capital city and all admin cities that are
        always included.

    candidates_gdf: gpd.GeoDataFrame
        The remaining candidating cities to evaluated, sorted by
        descending population.

    min_spacing_m: float
        Minimum distance in meters between two selected cities.

    max_cities: int
        Upper limit on the wanted number of selected cities.

    Returns
    -------
    gpd.GeoDataFrame
        A combined GeoDataFrame of predefined cities and the other
        valid candidating cities.
    """

    selected_cities_gdf = seed_gdf.copy()

    for index, city in candidates_gdf.iterrows():
        distances = selected_cities_gdf.distance(city.geometry)
        min_distance = distances.min()

        if min_distance >= min_spacing_m:
            selected_cities_gdf = pd.concat(
                [selected_cities_gdf, city.to_frame().T],
            ignore_index=True)

        if len(selected_cities_gdf) == MAX_CITIES:
            break

    return selected_cities_gdf
```

c) Running the Selection Loop

```
selected_cities_gdf = select_cities_by_space(
    seed_gdf = primary_cities,
    candidates_gdf = other_cities,
    min_spacing_m = MIN_SPACE_KM * 1000,
    max_cities =  MAX_CITIES,
)

assert len(selected_cities_gdf) > 0, "The dataset is empty."
print(f"Total cities selected: {len(selected_cities_gdf)}")
```

```
Total cities selected: 40
```

3. Export of processed Data

```
selected_cities_gdf.to_csv(PROCESSED_PATH, index=False)

print(f"Saved {len(selected_cities_gdf)} records to {PROCESSED_PATH}")
```

```
Saved 40 records to ..\data\processed\selected_cities_canada.csv
```

Now all relevant Cities for the analysis are selected and can be visualized. Therefore open now the visualization.ipynb notebook and follow the code to combine the fire data and the cities in an interactive map